



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO RIO GRANDE
DO NORTE
CAMPUS NATAL - ZONA LESTE
CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

DOUGLAS DE ARAÚJO ROCHA
LUCAS APRÍGIO DA SILVA
MESSEJANA ISLA CASSIANO
PEDRO LUCAS PEREIRA DE MEDEIROS
TALISE GONÇALVES BATISTA

RELATÓRIO DA QUARTA AVALIAÇÃO DO PROJETO INTEGRADOR

NATAL-RN
2026.1

INTRODUÇÃO

O Fox System, nome designado à aplicação desenvolvida, é um sistema web idealizado para otimizar a gestão e a interação entre alunos e instrutores da academia Fox Fitness. Após as etapas de concepção da interface com foco na estratégia mobile-first (front-end) e da construção de uma arquitetura robusta de dados e regras de negócio (back-end), a quarta etapa do Projeto Integrador concentra-se na integração definitiva destas camadas.

O objetivo principal desta fase foi consolidar a comunicação entre o front-end, desenvolvido em React/Next.js com TailwindCSS, e a API Restful do back-end, construída em Python utilizando o ecossistema Django e Django Rest Framework (DRF), garantindo a persistência eficiente em um banco de dados MySQL e fornecendo um sistema completo, seguro e responsivo.

INTEGRAÇÃO FRONT-END E BACK-END

A comunicação estrutural entre a interface do usuário e o servidor foi estabelecida através de requisições HTTP assíncronas. No lado do cliente (Next.js), a integração foi realizada utilizando funções para o consumo da API. Os dados inseridos nos formulários de autenticação ou nas buscas por alunos são empacotados e enviados ao back-end. A resposta do servidor é interceptada pelos Client Components, que atualizam o estado da tela em tempo real, dispensando recarregamentos desnecessários. Abaixo está a arquitetura adotada para a integração de endpoints:

POST /api/auth/login/: Autenticação de usuários (Alunos e Funcionários).

GET /api/clientes/: Listagem de todos os clientes cadastrados.

POST /api/clientes/: Criação de um novo cliente (Create).

GET /api/clientes/<id>/: Consulta de dados de um cliente específico (Read).

PATCH /api/clientes/<id>/: Atualização de dados cadastrais de um cliente (Update).

DELETE /api/clientes/<id>/: Exclusão de um cliente do sistema (Delete).

GET /api/funcionarios/<id>/: Consulta de dados do instrutor.

PATCH /api/funcionarios/<id>/: Atualização de perfil do instrutor (ex: foto de perfil).

POST /api/instrutores/vincular/: Vinculação de um aluno à carteira de um instrutor.

GET /api/treinamentos/: Listagem de templates únicos de treino disponíveis.

GET /api/treinios/<id>/: Consulta de um treino específico.

PATCH /api/treinios/<id>/: Atualização dos exercícios de uma ficha de treino.

POST /api/clientes/<id>/treinios/: Criação de uma ficha de treino em branco exclusiva para o aluno.

CONTROLE DE ROTEAMENTO E SEGURANÇA

Um dos pilares da integração foi a consolidação do sistema de controle de acesso baseado em papéis (Role-Based Access Control). O fluxo de segurança inicia-se no AuthLayout compartilhado, onde a autenticação define o comportamento das rotas através do useRouter.

- **Visão do Aluno:** O aluno é redirecionado para um dashboard focado no consumo de seus próprios treinos, sem capacidade de alterar regras do sistema. Informações sensíveis e avaliações físicas estão isoladas para visualização própria.
- **Visão do Instrutor:** O profissional possui acesso a uma barra de pesquisa assíncrona que consome o banco de dados e retorna apenas os alunos sob sua tutela, impedindo a sobreposição de dados entre diferentes professores.

VALIDAÇÃO DE FLUXO E DEBUGGING FINAL

Para garantir a estabilidade do produto final, a equipe conduziu testes de sistema simulando o fluxo completo de uso. Os testes de usabilidade previamente aplicados no front-end foram expandidos para incluir a validação de respostas do servidor sob diferentes condições.

A rotina de tratamento de erros no Next.js garante que falhas de conexão sejam reportadas de forma limpa, como exemplificado na interceptação padrão de erro:

```
try {
  const response = await fetch('/api/auth/login', {
    method: 'POST',
    body: JSON.stringify(credenciais)
  });
  if (!response.ok) throw new Error('Credenciais Inválidas');
  // Redirecionamento de sucesso
} catch (error) {
  setErrorMessage(error.message);
}
```

Com isso, confirmou-se que elementos visuais mantêm a legibilidade em dispositivos móveis durante o carregamento de dados e que tentativas de inserção de dados inválidos sejam devidamente interceptadas.

CONCLUSÃO

O desenvolvimento do Fox System representou a aplicação prática e integrada de múltiplos conceitos da engenharia de software, desde a concepção de mockups de alta fidelidade até a implementação de uma arquitetura limpa no back-end. A separação de responsabilidades (front-end independente do back-end) permitiu um trabalho colaborativo e gerou um sistema altamente escalável. O produto alcançado atende com êxito à proposta inicial: promover a inclusão digital no ambiente de academias, proporcionando uma ferramenta segura e intuitiva.